

TECHNICAL REFERENCE V4.2

Shokunin 職人 AI Engineering Ecosystem

38 skills, ChromaDB memory, 3 MCP servers, 4 subagents.

Cross-platform (Windows + Linux). Multi-runtime (OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf). Zero cost, open source.

Elias Oulkadi

Version 4.2 · May 2026

github.com/EliasOulkadi/shokunin

| Contents

01 What is Shokunin?

02 Architecture & Design

03 The 38 Skills

04 Memory System v4.2

05 MCP Servers

06 Subagents

07 Cross-Platform Setup

08 Multi-Runtime Support

09 Workflow Examples

10 CI/CD & Quality

11 Quick Reference

| What is Shokunin?

Shokunin (職人, Japanese for artisan) is an open source ecosystem that transforms AI coding agents into production-grade engineering workstations. **38 domain-specific skills, persistent ChromaDB memory, and cross-platform support** eliminate the blank-slate problem that plagues every AI session.

Instead of starting every session with zero context, Shokunin provides a complete engineering environment: domain expertise through auto-activating skills, persistent context across sessions through a three-layer memory system, direct system access through MCP servers, and automated maintenance through platform-native scheduling.

Everything runs on your machine. No cloud dependencies, no telemetry, no subscriptions. Works on Windows and Linux, across six different AI coding runtimes.

One command install:

Windows: `irm`

`https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1 | iex`

Linux: `bash <(curl -sL`

`https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh)`

| Architecture & Design

Five independent layers that each serve a specific role. Every layer can be replaced, extended, or run on a different operating system without affecting the others.

Layer	Components	Cross-Platform?
Interface	OpenCode, Claude Code, Cline, Cursor, Continue.dev, Windsurf	Yes
Agent	38 skills, superpowers plugin, 4 subagents	Yes
Infrastructure	3 MCP servers, ChromaDB	Yes
AI Providers	NVIDIA, OpenAI, Anthropic, Ollama, DeepSeek	Yes
System	PowerShell/bash scripts, Task Scheduler/crontab	Platform-specific

The key design insight: the core (skills, memory, MCP) is fully cross-platform. Only the wrapper scripts and scheduling are platform-specific, keeping the architecture clean and maintainable.

Key Stats

- 42 commits in the repository
- 38 skills covering infrastructure, backend, frontend, mobile, quality, content, productivity
- 8 PowerShell scripts for Windows, 6 bash scripts for Linux, 2 Python scripts (cross-platform)
- 6 PDF guides in the docs directory
- 4 CI jobs testing Python, Windows, Linux, and configuration
- 8 ChromaDB entries of persistent memory

| The 38 Skills

Every skill is a SKILL.md file with trigger-optimized descriptions, procedural workflows, error handling tables, production checklists, anti-patterns, and cited sources. They auto-activate when the agent matches your description.

By Domain

Domain	Skills	Count
Infrastructure	docker, kubernetes, terraform, ci-cd, db-admin, runbook-gen	6
Backend	auth-architect, api-forge, db-sculptor, error-handler	4
Frontend	component-forge, responsive-engine, motion-craft, landing-craft, ui-ux-pro-max	5
Mobile	flutter, react-native	2
Quality	test-commander, performance-profiler	2
Content & Business	communication, content-marketing, business-proposals, seo-geo, translate-craft, documentation, design	7
Productivity	git-workflow, windows-powershell, strategy, runbook-gen, finance, legal-counsel, whendone-plus, chromadb, memory	9
Documents	kami, portfolio-auto	2
System	cleanup-surgeon	1

Skill Quality

Every skill follows a strict format: YAML frontmatter, procedural workflow, error handling, production checklist, anti-patterns, sources. After the v4.2 quality pass, 31 out of 38 skills score 9/10 or higher. The remaining 7 have been upgraded with missing sections and now score 7/10+.

| Memory System v4.2

Three layers ensure no context is lost between sessions. ChromaDB provides semantic search; a parallel checkpoint timer saves every 5 minutes; the wrapper captures terminal output on exit.

Layer 1: Agent-Driven Saves

The agent saves context during the session using `chroma-helper.py` directly (no MCP dependency). Every 3-5 interactions a checkpoint is saved. After each decision, file change, or command execution, the agent saves the context.

Layer 2: Parallel Checkpoint Timer

Every 5 minutes, a background process saves a timestamped checkpoint. This runs regardless of whether the agent saves anything. If the session crashes, at least the checkpoints remain.

Layer 3: Terminal Capture on Exit

When the session ends, the wrapper reads the terminal output. On Windows it reads the console buffer (5000 lines). On Linux it uses the `script` command (unlimited). The output is stripped of ANSI codes and saved.

Storage

What	Path	Format
Vector database	~/.shokunin/memory/chroma_db/	ChromaDB + sqlite3
Session summaries	~/.shokunin/memory/sessions/	Markdown + JSON
Active session data	~/.shokunin/current-session.json	JSON
Cleanup logs	~/.shokunin/logs/cleanup.log	Plain text

Data Format

```
{
  "type": "decision | file | command | checkpoint | session_end",
  "timestamp": "2026-05-14T10:30:00Z",
  "project": "my-project",
  "tags": ["auth", "decision"],
  "session_id": "session-20260514-103000-A7F3",
  "text": "Descriptive context of what happened"
}
```

| MCP Servers

MCP (Model Context Protocol) servers give the AI agent direct system access. They start on demand, consume no resources when idle, and work identically on Windows and Linux.

Server	Tools	Ondemand?
filesystem	read, write, edit, search, list files	Yes (npx)
fetch	fetch web pages and APIs	Yes (npx)
memory	store_context, search_context, get_session_summary	Yes (python)

The memory MCP server is 317 lines of Python implementing JSON-RPC 2.0 over stdin/stdout. It logs all operations and supports typed entries with project and tag filtering.

| Subagents

Four specialized subagents run in parallel with their own models. They handle code review and debugging without blocking the main conversation.

Subagent	Model	Fallback	Purpose
code-review	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Bug + security review
code-review-local	Ollama Qwen3 14B	-	Offline code review
debugger	NVIDIA Kimi K2.6	Ollama Qwen3 14B	Root cause analysis
debugger-local	Ollama Qwen3 14B	-	Offline debugging

Cross-Platform Setup

The same skills, memory, MCP servers, and subagents work on both Windows and Linux. Only the wrapper scripts, scheduler, and profile configuration differ.

Windows vs Linux Comparison

Capability	Windows	Linux
Terminal capture	Buffer API (5000 lines, PS5.1)	script command (unlimited)
Wrapper	run-opencode.ps1	run-opencode.sh
Installer	install.ps1 (517 lines)	install.sh (206 lines)
Profile shadow	PowerShell \$PROFILE	.bashrc/.zshrc
Scheduling	Task Scheduler	crontab
Healthcheck	memory-healthcheck.ps1	memory-healthcheck.sh
Cleanup	scan-cleanup.ps1	scan-cleanup.sh
Tests	test-memory.ps1	test-memory.sh
Transcript parser	read-transcript.ps1	read-transcript.py

The Linux version is not a second-class port: it has advantages like unlimited terminal capture and simpler bash scripts. The shared Python core (chroma-helper.py, mcp-server.py) detects the OS automatically for path resolution.

| Multi-Runtime Support

The ecosystem is not tied to OpenCode. Skills, memory, MCP servers, and scripts work across six different AI coding runtimes.

Runtime	Skills	Memory	MCP	Scripts	Config File
OpenCode	Native	Native	opencode.json	.ps1 + .sh	Built-in
Claude Code	Reads SKILL.md	Via MCP	claude-code.json	.ps1 + .sh	Template
Cline	Reads SKILL.md	Via .clinerules	cline-settings.json	.ps1 + .sh	Template
Cursor	Reads SKILL.md	Via .cursor/rules	cursor-mcp.json	.ps1 + .sh	Template
Continue.dev	Reads SKILL.md	Via .continue/rules	continue-config.yaml	.ps1 + .sh	Template
Windsurf	Reads SKILL.md	Via .windsurf/rules	Via UI	.sh only	Template

Template configs are in `.pack/templates/` and runtime rules in `.pack/rules/`. No existing files are modified to add a runtime -- everything is additive.

| Workflow Examples

Real scenarios showing how the ecosystem works end-to-end. Describe your task in natural language -- skills auto-activate.

Building Authentication

User: "Add Google OAuth login to my Express app"

1. auth-architect activates: OAuth2 + PKCE flow, JWT rotation, session management
2. db-sculptor activates: User schema with Prisma, migration, indexes
3. error-handler activates: Structured error handling for auth routes
4. test-commander activates: Integration tests with mocked OAuth
5. Memory checkpoint: "Added Google OAuth with PKCE, RS256 signing"

Deploying to Production

User: "Dockerize and deploy to Kubernetes"

1. docker activates: Multi-stage Dockerfile, BuildKit cache, compose
2. kubernetes activates: Deployment, Service, Ingress, HPA, PDB
3. ci-cd activates: GitHub Actions pipeline with healthcheck
4. db-admin activates: PostgreSQL backup strategy

Decluttering the System

User: "Clean up old files I don't need"

1. cleanup-surgeon activates: runs scan-cleanup.ps1 or scan-cleanup.sh
2. Scans Desktop, Downloads, TEMP for files older than 7 days
3. AI analyzes: classifies as safe/review/keep
4. Moves safe files to Recycle Bin (Windows) or trash (Linux)
5. Reports: "1.4 GB recovered, 1988 files cleaned"

| CI/CD & Quality

GitHub Actions validates every component on every push. Four parallel jobs complete in approximately 2 minutes.

Job	OS	Validates
test-python	Ubuntu	ChromaDB import, chroma-helper save/search, MCP tools, MCP store/search
test-scripts	Windows	PowerShell syntax (all .ps1 files), chroma-helper on Windows
test-linux	Ubuntu	Bash syntax (all .sh files), cross-platform paths, install.sh parsing
validate-config	Ubuntu	JSON validity, SKILL.md required fields, CLAUDE.md memory section

All scripts follow senior engineer coding standards: guard clauses, named constants, error handling with retry logic (2 attempts before fallback). Security measures include: sed delimiter guard for API keys, OWASP input validation, and principle of least privilege.

Quick Reference

Key Paths

What	Path
Skills	~/.config/opencode/skills/ (38 SKILL.md files)
Memory data	~/.shokunin/memory/chroma_db/
Scripts	~/.shokunin/scripts/ (PowerShell + Python + bash)
Active session	~/.shokunin/current-session.json
GitHub repo	github.com/EliasOulkadi/shokunin

Quick Commands

Action	Windows	Linux
Start session	opencode	opencode
Search memory	search-memory.ps1 -Query "X"	python chroma-helper.py search "X"
Save context	save-memory.ps1 -Text "X"	python chroma-helper.py save "X" ...
Healthcheck	.\memory-healthcheck.ps1	./memory-healthcheck.sh
Run tests	.\test-memory.ps1	./test-memory.sh
Clean files	.\scan-cleanup.ps1 -Scan	./scan-cleanup.sh
Git status	gst	gst
Git commit	gc "message"	gc "message"
npm install	ni	ni

Links

Repo: github.com/EliasOulkadi/shokunin

Install (Windows): irm

<https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.ps1> | iex

Install (Linux): bash <(curl -sL

<https://raw.githubusercontent.com/EliasOulkadi/shokunin/master/install.sh>)

License: MIT

Created by Elias Oulkadi

Version 4.2 | May 2026 | 42 commits | 38 skills | 8 scripts | 6 PDFs